Name: Javaughn Stevens
Date: March 7, 2026
M#: M21185543

1. What did you learn?

I learned about the various types of hashing methods in Java (which might also apply to other programming languages like C++ or Python), and how they can change multiple different values (keys) using an algorithm which can be applied to a variety of real world contexts: passwords, cybersecurity, reference books, etc. I also saw this lab as a connection of the mathematical side of programming and how programs might use given algorithms in a real program implementation.

2. What challenges did you encounter, if any?

I initially struggled with understanding how hashing works even after watching the provided videos, but I eventually figured it out after working out the problems on the slides. I also realized during this process that manually hashing usually leads to making arithmetic mistakes due to processing different numbers from different categories (hashed values, indexed values, keys, the algorithm, using the algorithm multiple times for a key due to collisions, remembering which indexes were filled previously, etc), which is not the case in a real program where the computer handles all of this.

Screenshots of Hash Tables From Slides:

- Create a hash table using open addressing with linear probing

N = 13

Linear Probing

... go down one

If you reach then end, go back up to index 0

Array:					
0					
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					

Array:	Keys	Hashed Values	Index Values	Please note: I added additional keys to	
0	15	13	2	completely fill the array.	
1	54	27	3	15 % 13 = 2	
2	13	15	0	54 % 13 = 2 (Collision!)	2 + 1 = 3
3	10	54	10	13 % 13 = 0	
4	135	24	5	10 % 13 = 10	
5	114	135	11	135 % 13 = 5	
6	49	174	12	114 % 13 = 10 (Collision!)	10 + 1 = 11
7	174	89	6	49 % 13 = 10 (Collision!)	Indexes 10, 11 are full, so index is 12
8	27	64	1	174 % 13 = 5 (Collision!)	5 + 1 = 6
9	24	90	4	27 % 13 = 1	
10	89	10	7	24 % 13 = 11	Indexes 11 and 12 are full, as well as indexes 0-3, so index = 4
11	64	114	8	89 % 13 = 5 (Collision!)	Indexes 11 and 12 are full, as well as indexes 0-6, so index = 7
12	90	49	9	64 % 13 = 4 (Collision!)	Index 12 as well as Indexes 0-7 are full, so index = 8
				90 % 13 = 6 (Collision)	Index 12 as well as Indexes 0-8 are full, so index = 9
Linear Probing	h(x) = x % 13			Keys: 15, 54, 13, 10, 135, 114, 49	
				174, 27, 24, 89, 64, 90	

- Using open addressing with quadratic probing

N = 13

Quadratic Probing

... try 1 away from where it should be inserted

if still full try 4 away from where it should be inserted

if still full try 9 away from where it should be inserted

then 16, 25... etc. These are perfect squares. Don't count base when counting!

Array:					
0					
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					

Array:	Keys	Hashed Values	Index Values		
0	28	39	2	28 % 13 = 2	
1	93	66	3	93 % 13 = 2 (Collision!)	2 + 1^2 = 3
2	96	28	5	96 % 13 = 5	
3	59	93	7	59 % 13 = 7	
4	192	53	10	192 % 13 = 10	
5	45	96	6	45 % 13 = 6	
6	39	45	0	39 % 13 = 0	
7	11	59	11	11 % 13 = 11	
8	87	100	9	87 % 13 = 9	
9	66	87	1	66 % 13 = 1	
10	53	192	4	53 % 13 = 1	1 + 1 = 2 (full), 1 + 4 = 5 (full), 1 + 9 = 10 (full), 1 + 16 = 17
11	90	11	12	90 % 13 = 12	1 + 16 = 17, 17 % 13 = 4
12	100	90	8	100 % 13 = 9	(must go to index 8, last available element)
				Keys: 28, 93, 96, 59, 192, 45, 39	
Quadratic Probing	h(x) = x % 13			11, 87, 66, 53, 90, 100	

- Using open addressing (division hashing) and the linear-quotient collision path algorithm

N = 13, 4k+3 prime = **19**

Array:

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	

Array:	Keys	Hashed Values	Index Values
0	18	99	5
1	43	85	4
2	67	67	2
3	69	120	11
4	20	13	7
5	28	18	12
6	50	50	6
7	85	20	1
8	120	93	3
9	36	150	10
10	93	36	8
11	150	69	9
12	99	28	0
Division Hashing	h(x1) = x % 13		
LQ Hashing (Double)	h(x2) = 19 - (x % 19)		
formula is from text	(hx1 + ihx2) % 13		
	i = 0, 1, 2, ...		

LQHashing:

1. i_p = pk % N
2. q=pk/N
   if (q%N != 0)
      offset = q
   else
      offset = 4k+3 prime
3. While collisions:
      i_p = (i_p + offset) % N
4. Set Array[i_p]=key

18 % 13 = 5	
43 % 13 = 4	
67 % 13 = 2	
69 % 13 = 4 (Collision!)	19 - (69 % 19) = 7 4 + 7 = 11
20 % 13 = 7	
28 % 13 = 2 (Collision!)	19 - (28 % 19) = 10 2 + 10 = 12
50 % 13 = 11 (Collision!)	19 - (50 % 19) = 7 11 + 1 * 7 = 18 (index 5), 5 % 13 = 5 (full), 11 + 2 * 7 = 25 25 % 13 = 12 (full) 11 + 3 * 7 = 32 % 13 = 6
85 % 13 = 7 (Collision!)	19 - (85 % 19) = 10 7 + 10 = 17 (index 4) 4 % 13 = 4 (full), 7 + 1*10= 17 (index 4, full), 7 + 2*10 = 27 % 13 = 1
120 % 13 = 3	
36 % 13 = 10	
93 % 13 = 2 (Collision!)	19 - (93 % 19) = 2 2 + 1 * 2 = 4 (full), 2 + 2 * 2 = 6 (full), 2 + 3 * 2 = 8 8 % 13 = 8
150 % 13 = 7 (Collision!)	19 - (150 % 19) = 2 2 + 1 * 7 = 9
99 % 13 = 8 (Collision!)	index 0 is last available spot
Keys: 18, 43, 67, 69, 20, 28, 50	
85, 120, 36, 93, 150, 99	

- Bucket hashing where (N=10) and ip = (p_k) % N

Array:

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Array:	Keys	Hashed Values	Index Values		
0	75	30, 60	5	75 % 10 = 5	
1	48	91	8	48 % 10 = 8	
2	88	null	8	88 % 10 = 8 (Collision!)	
3	30	null	0	30 % 10 = 0	
4	60	null	0	60 % 10 = 0 (Collision!)	
5	78	75, 65	8	78 % 10 = 8 (Collision!)	
6	91	46	1	91 % 10 = 1	
7	46	null	6	46 % 10 = 6	
8	65	null	5	65 % 10 = 5 (Collision!)	59 % 10 = 9
9	59	59	9	Keys: 75, 48, 88, 30, 60, 78, 91, 46, 65, 59	
ip = h(x) = x % 10					

- Repeat what you have done by using an open addressing algorithm of your choice
  - Come up with your own 15 keys and a 21-element array
  - Challenge yourself using an algorithm that you are most **unfamiliar** with to maximize your learning experience

  Elements: 15

  N = 21

Array:	Keys	Hashed Values	Index Values		
0	7		7	7 % 21 = 7	
1	23	64	2	23 % 21 = 2	
2	45	23	3	45 % 21 = 3	
3	92	45	8	92 % 21 = 8	
4	56	88	14	56 % 21 = 14	
5	101	5	17	101 % 21 = 17	
6	12	null	12	12 % 21 = 12	
7	88	21	4	88 % 21 = 4	
8	33	92	13	33 % 21 = 12 (Collision!)	12 + 1 = 13
9	5	null	5	5 % 21 = 5	
10	75	null	16	75 % 21 = 12 (Collision!)	12 + 1 = 13 12 + 4 = 16
11	64	null	1	64 % 21 = 1	
12	18	12	18	18 % 21 = 18	
13	120	33	15	120 % 21 = 15	
14	3	56	19	3 % 21 = 3 (Collision!)	3 + 1 = 4 3 + 4 = 7 3 + 9 = 12 3 + 16 = 19
15	null	120	null		
16	null	75	null		
17	null	101	null		
18	null	18	null		
19	null	3	null		
20	null	null	null		
				Keys: 7, 23, 45, 92, 56, 101, 12,	
				88, 33, 5, 75, 64, 18, 120, 3	
h(x) = x % 21					